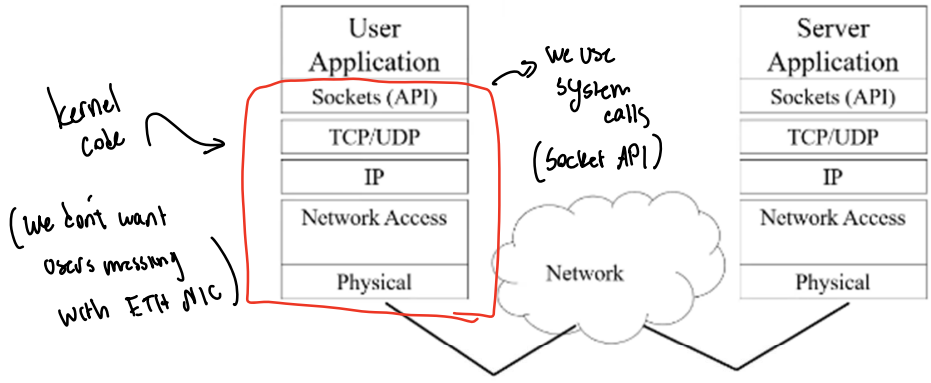
Socket Programming

Socket Programming

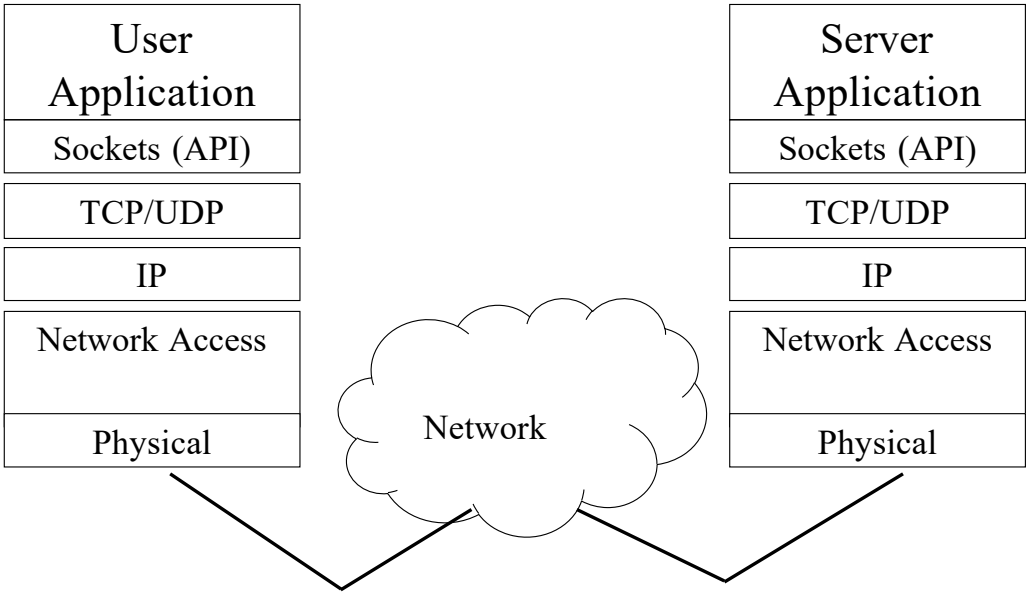
(how to write programs that talk over the network)



API = Application Programmins Interface
(set of functions we use to access resources)

System call → partly in user, partly in kernel
trap to kernel, let kernel handle it.

Socket Programming



Connection oriented:
- setup ⇝ connect() / accept()
- use ⇝ send() / recv()
- teardown ⇝ close()

TCP – Client/Server

touch on UDP, but we won't spend much time

Client (active) Side

1. Create a **socket()** (file descriptor)

blocking

2. **connect()** to the server

Server (passive) Side

server runs 1st (waiting for client)

1. Create a **socket()** (file descriptor)
2. Configure server socket
   - **bind(), listen()**
3. Wait on Client to connect
   - **accept()** (blocking)

setup

3. **send()/recv()** data ⟷ 4. **send()/recv()** data

use

4. **close()** the connection ⟷ 5. **close()** the connection

teardown

socket() Call – Create the socket File Descriptor

↳ what abstraction do we know about → files!

- **File Access - Open a file** using: <0 on error

  file descriptor = open ( )

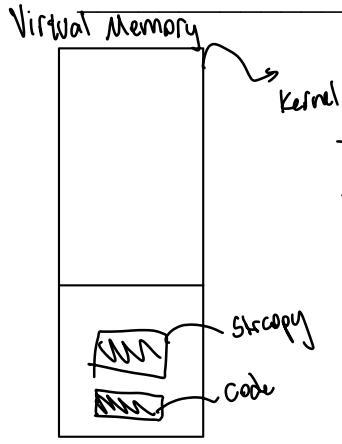
  - open() creates a structure in the kernel to track/manage the file I/O
  - Use **read() and write()** to input and output to file

- **Sockets - Open a socket** using:

  file descriptor = socket (_____); → check for <0

  - socket() creates a structure in the kernel to track/manage network I/O
  - To input and output to a socket use:
    - recv(…) and send() or recvfrom(…) or sendto(…)

Creating a Socket/Opening a file



Syscall → jumps into kernel (not your memory)

- won't segfault, syscall stops, returns -1
- check system calls for errors (is it working in kernel)
  - less than zero?
    - perror(" ")
    - exit(-1)

read from stdin?

- system call
  - 0 → standard in
  - 1 → standard out
  - 2 → standard err

Types of Sockets – TCP and UDP

- **Transmission Control Protocol (TCP)**
  - Is a **connection-oriented** service.
  - TCP is a **reliable** service (all data in order, error free)
  - It uses a **byte stream** in order to send data.
  - Socket Programming: **SOCK_STREAM** specify TCP/UDP

- **User Datagram Protocol (UDP)**
  - Is a **connectionless** service.
  - UDP is a **best effort** service (drop it on the network and forget about it)
  - It uses **individual message transfers** to send data.
  - Socket Programming: **SOCK_DGRAM**

socket() System Call

int **socket**(address_family, type, protocol);

AF_INET6 (always) - supports IPv4/v6

SOCK_STREAM

0

- **Address Family**
  - **AF_INET6**
    - Supports both IPv6 and IPv4
    - Use AF_INET4 for IPv4 only

- **Type** of communications
  - TCP use **SOCK_STREAM**
  - UDP use **SOCK_DGRAM**

- **Protocol** = 0 (means use default)

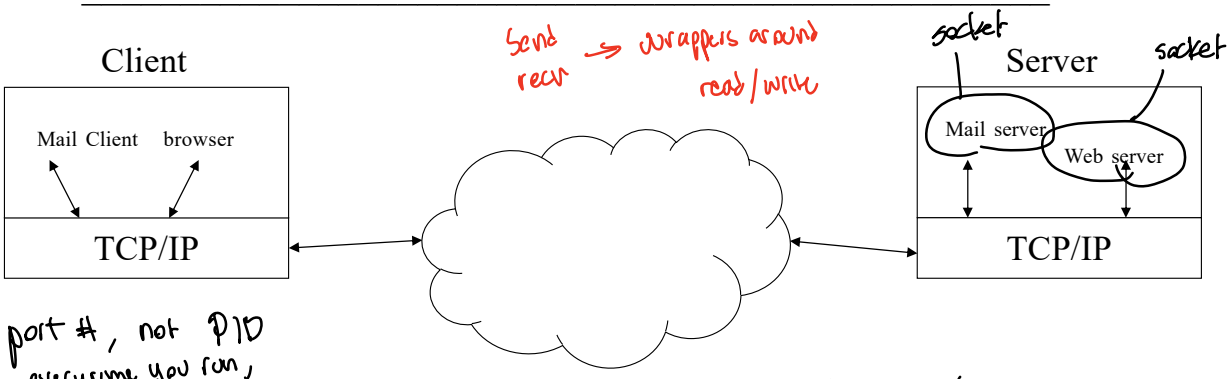
socket() System Call

```
if ((socketNumber = socket(AF_INET6, SOCK_STREAM, 0)) < 0)
{
    perror("socket call");
    exit(-1);
}
```

NOTE: A socket (a file descriptor) is only used and known by the current process

- **Problem:**
  - How can a process on one machine send a packet to the correct process on a remote machine?

Naming a Socket - Uniquely identify the Server End Point



send recv → wrappers around read/write

port #, not PID
everytime you run, you get different PID

Socket is local to process, no one knows about you

- What info is needed for a client to **Identify** a server?

socket = local port = global bind socket → port

port # = number maintained by kernel
only one person can have it

1) protocol family [AF_INET6]
2) IP address of the server
3) Port # the server is using

"naming the socket"

TCP Port Numbers – Used to identify a Communications Endpoint

- **Port Numbers**
  - Are maintained by the protocol stack (Operating system)
  - All sockets (both client and server) will have a port number
  - Obtaining a port number
    - Explicitly: ⇝ ask for one
      - server calls bind()
        - needs to know
    - Implicitly:
      - if you send on a socket w/o port #
        - one is assigned by kernel
          - client doesn't care about port #

really every packet.
- client / server IP
- client / server port

Naming a socket using the **bind()** System Call

- **Naming a socket:**
  - To name the Socket use the bind() system call:

bind(socket, name, name length); <0 on error

↑commonly wrong

- **Using bind() - Two ways to ask for a port number:**
  - explicitly
  - implicitly
- Some port numbers are **reserved** (see /etc/services)
  - how does google become port ⇝ web traffic = port 80
- bind() only needs to be called on the server

Using the **bind()** System Call

socket we wanna name

name (IP addr, family, port #)

length of info

```
#include <sys/socket.h>

int bind(int socket, const struct sockaddr * myaddr, socklen_t addrlen)
```

- socket is the socket number returned from call to socket(…)
- myaddr is a pointer to the address structure containing the IP address, port number and address family
- addlen = the length of the address field - sizeoff(sockaddr)
- bind() returns < 0 on Error (it's a system call!!!!)

Setting up a call to Bind

Address Structure

AF_INET6

```
struct sockaddr_in6 {
        sa_family_t     sin6_family;   /* AF_INET6 */
        in_port_t       sin6_port;     /* port number */
        uint32_t        sin6_flowinfo; /* IPv6 flow information */
        struct in6_addr sin6_addr;     /* IPv6 address */
        uint32_t        sin6_scope_id; /* Scope ID (new in 2.4) */
};
```

can't hardcode IP addresses

inbaddr_any = any IP ports, connect

```
// Now NAME the socket
server.sin6_family= AF_INET6;
server.sin6_addr = in6addr_any;   //wild card machine address
server.sin6_port= htons(portNumber);
```

0 means give us random port #

bind() call example

```
int socket_num = 0;
struct sockaddr_in6 address;

if ((socket_num = socket(AF_INET6, SOCK_STREAM, 0)) < 0)
{
        perror("socket call");
        exit(-1);
}

// Now NAME the socket
server.sin6_family= AF_INET6;
server.sin6_addr = in6addr_any;   //wild card machine address
server.sin6_port= htons(portNumber);

if (bind(socket_num, (struct sockaddr *) &address, sizeof(sockaddr) < 0))
{
        perror("bind call");
        exit(-1);
}
```

use of socket call to setup TCP socket

assigning struct info to socket

TCP – Client/Server

Client (active) Side

1. Create a **socket()** (file descriptor)

Server (passive) Side

1. Create a **socket()** (file descriptor)
2. Configure server socket - **bind(), listen()**

gets everything ready

2. **connect()** to the server
3. Block on Client to connect - **accept()**
3. **send()/secv()** data
4. **send()/recv()** data
4. **close()** the connection
5. **close()** the connection

Connection Establishment - Connect, Accept, Listen

- Stream sockets (TCP) must establish a connection before data can be transferred

- Client – Server Architecture
  - Server needs to **accept** new connections
  - Client is responsible for actively starting the connection to the server

  - Server → tells kernel → BTW im a server, buffer my data
    - listen() → called once
      accept() → servers waits for connect

  - Client
    - connect()

Server – listen() and accept() System Calls

- **Passive end (server)**
  - Server calls **listen()** once
  - The server waits on **accept()** for a connect() from the client(s)

→ how many to buffer (128 is linux max)

- **int listen(int socket, int backlog)**
  - backlog = Maximum queue length for pending (un-accepted) connections
  - Needed to be called (ONCE) to allow connections to be accepted
  - Linux configured max defaults to 128

- **int accept(int socket, struct sockaddr * client_addr, socklen_t * addrlen)**
  - client_addr = the accept call will fill in the address of the client

→ accept establishes connection
→ blocking call until client connects

Client – connect() System Call

- **Active end (client)** – make connect() requests to server

int connect(int socket, const struct sockaddr *addr, socklent_t addrlen**)**

- **addr**
  - Is a pointer to the address structure
  - Identifies the server
    - family → AF_INET6
    - Server IP
    - Server Port #

connect() System Call

```
uint8_t * ipAddress = NULL;
struct sockaddr_in6 server;

// setup the server structure
server.sin6_family = AF_INET6;
server.sin6_port = htons(atoi(argv[2]));  // example argv[2] is port number

// get the address of the server
if ((ipAddress = getIPAddress6(serverName, &server)) == NULL)
{ … perror… exit..}
printf("server ip address: %s\n", getIPAddressString(ipAddress));
if(connect(socket_num, (struct sockaddr*)&server, sizeof(server)) < 0)
{
        perror("connect call");
        exit(-1);
}
```

setup server info

↑ uses DNS to translate (Domain Name Server)

DNS goes to multiple external DNS servers

TCP Connection

Client	**Server**
sk =socket()	s1=socket()
	bind(s1,…)
(note: need server port #)	getsockname(….)
	printf(… port …)
	listen(s1,…)
connect(sk,...) ⟷	s2 = accept(s1,…)
	(accept blocks waiting on a connect())
	(Could create another thread here)
send(sk…) ⟶	recv(s2,…)
	send(s2…)
recv(sk…) ⟵	
	close(s2)
close(sk) ⟷	
	(when server is shutting down)
	close(s1)

Setup

returns → socket

retrieves it from us the port #

get connection established

Server gives new socket number for every accept

Server with Multiple Clients

server

```
while(1){
    s2 = accept()
    fork() → child sends / receives using s2,
```

send / receive | send / receive

this is for handling many clients simultaneously

In program 2, one process

program 2: many FD for many sockets (open sockets)

another way is fork (new process, new virtual memory)

threads / pthreads

A UDP connection

UDP → no setup and teardown

server doesn't keep track of sockets

TCP tells you immediately if you screw up

Client		**Server**
		s1=socket()
		bind(s1,…)
sk =socket()		getsockname(….)
(note: need server port #)		printf(… port …)
sendto(sk, ….)	→	recvfrom(s1,…&from…)
recvfrom(sk,….)	←	sendto(s1, …from..)
close(sk)		close(s1)

The server picks up the client info (client IP and Port) via the 'from' paramater in the recvfrom() call

TCP Sending and Receiving Data

- TCP Send data:

  socket data data length

  int send(int socket_num, char * data, int data_len, int flags**)**

  ↳ for talking in TCP

  flag 0

- TCP Receive data:

  int recv(int socket_num, char * data, int data_len, int flags)

- Both functions return the amount of data sent/received.
- If return value is < 0 then an error occurred.

TCP – Sends/Recvs a stream of bytes

- TCP communications is data stream oriented → stream doesn't maintain boundaries from what you send
- Send **two** messages: **"Networks", "Rule"**

→ there is TCP configuration but if you receive at same time, kernel put them together

You may recv():

recv() call #1	recv() call #2	What happened
Networks	Rule	recv() as sent
Networks Ru	Le	Second send() split
Net	Works rule	First send() split
Networks Rule	(call blocks)	Both send()s combined

TCP buffers for 125 ms, then sends it

Running Servers and Clients

- **Run server first**
  - Server outputs its port number
  - ./myServer

    Outputs: Using port number: 54321

- **Next – Run the client**
  - Pass in the server machine name/IP and port number
    - ./myClient unix1.csc.calpoly.edu 54321
    - ./myClient 192.168.1.1 54321
  - Client and server on the same machine
    - ./myClient 127.0.0.1 54321
    - ./myClient localhost 54321